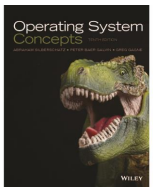


Operating Systems 2023

U3680

National Taipei University

Midterm Exam [Chapter 1 - Chapter 5]

---

1. What are the goals of **multi-programming** and **time-sharing**? (6%)
   - **Multi-programming: increase CPU utilization and keep CPU busy (3%)**
   - **Time-sharing: provide interactivity and reduce response time (3%)**

2. Please answer the following questions about **interrupts**:
   (a) What is the purpose of **interrupts**? (4%)
   Interrupts provide an event-driven way to change the flow of control in the CPU. (4%)
   (b) List **three** types of **software interrupts**. (6%)
   Invalid memory access, software error, system calls. (6%, 2% for each one)

3. Please answer the following questions about **system protection**:
   (a) How does an OS protect the **CPU resource** (for example, avoid an infinite loop of user programs)? (3%)
   Implement timers and use time-sharing. When the timer reaches the value 0, an interrupt occurs for the context switch. (3%, answering one of timer or time-sharing can get all scores)
   (b) How does an OS protect the **memory space** of a process? (3%)
   Use the base register and the limit register to restrict the memory address that a process can access. (3%)

4. Describe why **Application Programming Interface (API)** can improve **portability**. (4%)
   Different OSes share the same programming interface despite the underlying implementation or hardware being different. For this reason, when a program needs to be run on a different OS, it just needs re-compiling, not re-writing. (4%)

5. Why a **micro-kernel** approach is usually **safe** (3%) but has **poor performance** (3%)
   The micro-kernel approach keeps the Kernel as small as possible (fewer bugs). It is also safer because most parts of the original kernels are moved to user space. Thereby, a crash of these system programs will not lead to a crash of the entire system (3%). However, the communication between components requires message passing, which is slow. (3%)

6. **Message passing** and **shared memory** are two inter-process communication models. Please list their advantages and disadvantages as many as possible. (6%)
   - **Message passing**

  - **Strength: naturally synchronized (1%), can communicate across machines (1%)**
  - **Weakness: slower (1%)**
- **Shared memory**
  - **Strength: faster (1%)**
  - **Weakness: need to handle synchronization (1%), cannot communicate across machines (1%)**

7. Which of the following components of the program state are **shared across threads** in a multi-threaded process? (4%) Describe why the rest parts could **NOT** be shared. (4%)

   (a) Register values
   (b) Heap memory
   (c) Global variables
   (d) Stack memory

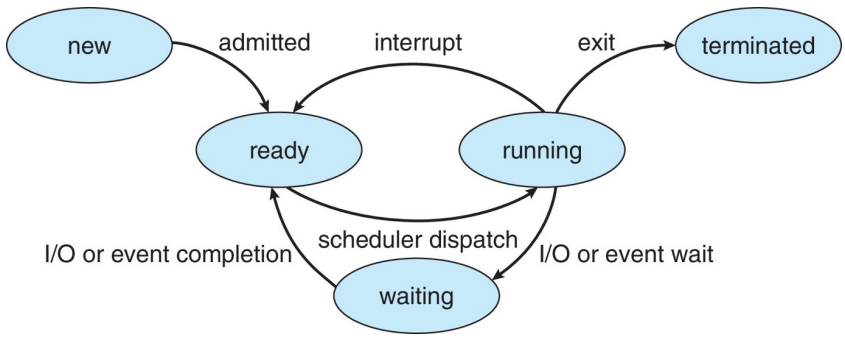
   - **b and c are shared (4%)**
   - **a and d could not be shared because different threads could run the different parts (functions) of a program (4%)**

8. Please answer the **two advantages** of **thread pools**. (6%)

   - **It is usually slightly faster to service a request with an existing thread than create a new thread. (3%)**
   - **It allows the number of threads in the applications to be bound to the size of the pool. (3%)**

9. Draw the diagram of the **process state** and briefly describe the meaning of each state. (10%)

   - **Diagram of process state (5%)**



   -
     - **New: the process is being created (1%)**
     - **Ready: the process is in the memory waiting to be assigned to a processor (1%)**
     - **Running: the process whose instructions are being executed by CPU (1%)**
     - **Waiting: the process is waiting for events (e.g., I/O) to occur (1%)**
     - **Terminated: the process has finished execution (1%)**

10. What are the problems when the **time quantum** of the **Round Robin** scheduling becomes too short and too long? (6%)

- **Too short: the context switch occurs too frequently, resulting in high overhead (3%)**
- **Too long: RR will become an FCFS scheduling algorithm, if a long process comes first, other processes would wait too long (3%)**

11. What is the difference between **preemptive** and **non-preemptive** scheduling? (8%)

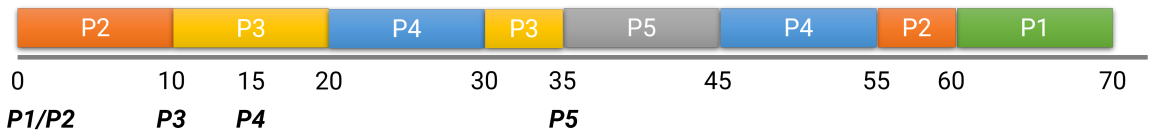
    - **Non-preemptive: the OS can not interrupt a process if it is under its CPU burst. CPU scheduling can only be performed when a running process is terminated or switched to the waiting state. (4%)**
    - **Preemptive: the OS can interrupt a process even if it is under its CPU burst. Besides a running process being terminated or switched to the waiting state, scheduling can also be performed when a context switch happens or a process is switching from the waiting state to the ready state. (4%)**

12. Describe why the original **shortest-job-first** scheduling is difficult to apply in real-world cases. (3%) How do we approximate it? (3%)
    We will never know the duration of the CPU bursts of processes until they are executed. (3%) We can only use the prediction of CPU bursts estimated from previous CPU bursts. (3%)

13. The following processes are being scheduled using a **preemptive**, **priority-based**, **round-robin** scheduling algorithm. Each process is assigned a numerical priority, with a **higher** number indicating a **higher** relative priority. The scheduler will execute the highest priority process. For processes with the same priority, a round-robin scheduler will be used with a time quantum of 10 units. **If a process is preempted by a higher-priority process, the preempted process is placed at the end of the queue**.

Process	Priority	Burst	Arrival
P_1	1	10	0
P_2	2	15	0
P_3	3	15	10
P_4	3	20	15
P_5	4	10	35

    (a) Show the scheduling order of the processes using a **Gantt chart**. (5%)



    (b) What is the **waiting time** for each process? (5%)

    - P_1: **60 (1%)**
    - P_2: **45 (1%)**
    - P_3: **10 (1%)**
    - P_4: **20 (1%)**
    - P_5: **0 (1%)**

14. Using the program shown below, what are the outputs of this program? (8%)

```c
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

#define SIZE  4

int nums[SIZE] = {0, 3, 6, 9};

int main()
{
int i;
pid_t pid;

   pid = fork();

   if (pid == 0) {
      for (i = 0; i < SIZE; i++) {
         nums[i] *= -i;
         printf("CHILD: %d ",nums[i]);
      }
   }
   else if (pid > 0) {
      wait(NULL);
      for (i = 0; i < SIZE; i++)
         printf("PARENT: %d ",nums[i]);
   }

   return 0;
}
```

- **line X: CHILD: 0 CHILD: -3 CHILD: -12 CHILD: -27 (4%, get 2% if the numbers are correct)**
- **line Y: PARENT: 0 PARENT: 3 PARENT: 6 PARENT: 9 (4%, get 2% if the numbers are correct))**